

Chapter 3 — Constitutional Kernel

Field	Value
Status	Revised after whole-book constitutional and cross-chapter audit
Version	0.2
Scope	Protected enforcement boundary, root state, kernel contracts, mutation protocol, and recovery authority
Constitutional basis	Decisions 1–7, 24–29, 35, 38–43, 47–51, 56, and 58
Owner review	No unresolved Owner decision after whole-book audit

1. Purpose

The Constitutional Kernel is the smallest HAL subsystem that must remain trustworthy for HAL to preserve constitutional identity. It is not a general-purpose application framework, planner, model host, or database. It holds the authority to establish root identity, evaluate protected policy, validate Owner authorization, preserve protected audit evidence, coordinate irreversible commits, and control recovery admission. Every other service is replaceable; the kernel is deliberately narrow, explicit, and continuously verifiable.

2. Design Boundary

The kernel enforces constitutional constraints. Governed services supply reasoning, proposals, evidence, state transformations, and execution results. The kernel decides only whether a protected request has the identity, authority, policy context, evidence, verification, and durable record required to proceed. It does not substitute its own judgment for the Intent, Trust, Planning, or Judgment services.

Kernel responsibility	Explicit non-responsibility
Root identity and trust anchor	Natural-language interpretation or personality
Protected policy and authorization evaluation	Planning, ranking, or goal selection
Owner Authorization Ceremony validation	Model inference or provider routing
Constitutional version and invariant admission	Search, retrieval, or knowledge graph maintenance
Protected audit append and evidence binding	Direct control of optional capabilities
Commit coordination for protected state	Application-specific business logic
Recovery and re-admission authority	Silent repair of constitutional state

3. Kernel Root State

Kernel root state is compact, versioned, cryptographically protected, and independently recoverable. It is sufficient to determine which constitutional identity is being restored and whether a proposed protected action is permitted. It never depends on a single model, provider, hardware host, or presentation layer.

Root-state record	Purpose
Constitutional Identity Record	Stable HAL identity, root public keys, approved recovery roots, and continuity metadata.
Owner Authority Record	Exactly one active constitutional Owner identity, protected succession state, and owner-authentication requirements.
Constitution and Invariant Registry	Signed constitutional versions, immutable invariant declarations, effective dates, compatibility markers, and amendment lineage.
Protected Policy Registry	Active constitutional and protected policies, policy version lineage, and policy-evaluator manifests.
Authorization Ceremony Registry	Pending and completed protected authorizations, exact change bindings, expiry, replay protection, and audit references.
Protected Audit Anchor	Append-only audit chain roots, checkpoints, witness references, and integrity-verification history.
Recovery Admission Record	Trusted recovery material, last verified restore state, emergency restrictions, and re-admission evidence.

4. Kernel Contracts

Kernel interfaces are typed, versioned contracts. A caller submits a command or query under an authenticated service identity. The kernel returns a signed decision, a narrowly scoped capability token, a denial with rationale, or a request for additional evidence or Owner authorization.

Contract	Primary input	Kernel result
EvaluateProtectedAction	Actor, intent, scope, policy context, evidence, requested effect.	Allow, deny, require verification, require Owner ceremony, or route to recovery/safe mode.
BeginProtectedTransaction	Authorized plan, rollback/compensation path, blast radius, verification requirement.	Transaction lease and immutable policy/authority binding.
CommitProtectedMutation	Transaction result, verification evidence, state version precondition, audit payload.	Atomic commit, compensating action, or durable pending/recovery state.

Contract	Primary input	Kernel result
ValidateOwnerAuthorization	Exact change identifier, current owner session evidence, fresh authorization factor.	One-time signed authorization or denial.
VerifyConstitutionalState	Constitution version, invariant set, root-state evidence, audit anchors.	Admit, restrict, or enter Safe Mode.
AdmitRecovery	Restore evidence, integrity report, reconciliation plan, current policy and identity state.	Recovery admission plan or rejection.
IssueDelegatedExecutionEnvelope	Authorized actor, capability scope, time limit, transaction/correlation context.	Non-transferable, auditable, narrowly scoped execution envelope.

5. Protected Mutation Protocol

A protected mutation is any state change that affects constitutional rules, protected policy, Owner authority, identity, trust requirements, authentication requirements, audits, quarantines, Treaties, capability classes, or protected permanent state. Protected mutations use a deliberate prepare–verify–authorize–commit sequence.

- **Prepare:** the requesting service creates a complete change object with exact intended effect, alternatives, affected state, risk, rollback or recovery plan, and required verification.
- **Validate:** the kernel checks identity, delegation, policy, constitutional version compatibility, transaction preconditions, current operating mode, and evidence freshness.
- **Verify:** the Verification Service supplies required simulation, shadow, canary, integrity, or recovery evidence; the kernel records its result without inventing confidence.
- **Authorize:** if the change crosses a protected threshold, the kernel requires the Owner Authorization Ceremony bound to the exact immutable change identifier.
- **Commit:** the kernel writes the protected audit record and commits only if state versions, authorization, verification, and policy bindings still match.
- **Observe:** outcome evidence, rollback readiness, and post-change health are attached. A changed proposal invalidates prior authorization and must restart the ceremony.

6. Owner Authorization Ceremony Enforcement

The kernel enforces the constitutional distinction between deliberative assent and verified authorization. Ordinary conversational agreement, including “yes,” “lock it in,” or “go ahead,” is never an execution credential for a protected change. The kernel accepts only a fresh, time-limited, non-replayable authorization bound to one exact change object and supported by current Owner identity and liveness evidence.

Requirement	Kernel enforcement
Exact binding	Authorization includes the immutable change identifier, affected records, and canonical change digest.
Freshness	Authorization expires quickly and cannot be reused after completion, timeout, or material proposal change.
Identity and liveness	The kernel verifies active Owner authentication

Requirement	Kernel enforcement
	evidence; a successful historical session is insufficient.
Factor independence	Authorization factor is separate from ordinary conversation and may combine passkey, hardware key, PIN, biometrics, signed Owner device, or future approved liveness method.
Auditability	The ceremony records the proposal, disclosures, authentication evidence class, authorization result, and execution outcome without unnecessarily exposing secrets.
Failure behavior	If any required check is unavailable or uncertain, the change remains pending or is denied; standards never relax because a factor is unavailable.

7. Invariants and Safe Mode

The kernel treats an invariant conflict, untrusted root state, broken audit chain, unresolved Owner identity conflict, or unauthorized constitutional mutation as a constitutional incident. It enters Safe Mode for affected protected domains, preserves evidence, and permits only inspection, recovery, and Owner-authorized repair paths.

- There is exactly one active constitutional Owner identity at any time.
- No protected state change is valid without a corresponding immutable audit record and applicable authorization evidence.
- No service may impersonate another service or broaden its delegated authority.
- No node, provider, presence, network location, or historical trust score substitutes for current identity, authorization, and policy evaluation.
- Failures may change capabilities but may never silently change HAL's rules.
- Constitutional identity persists through implementation replacement only when root state, constitutional lineage, governed memory, audit continuity, and authority are preserved.

8. Kernel Availability and Recovery

The kernel is replicated or recoverable according to the deployment's protected-state durability policy, but it is logically singular: concurrent partitions cannot independently make protected canonical commits. A partition lacking required quorum or current constitutional state enters Restricted mode. Recovery uses signed constitutional mirrors, protected audit checkpoints, independent backups, and reconciliation evidence before resuming protected work.

Condition	Kernel behavior
Optional governed service unavailable	Kernel continues enforcing protected controls; related capability is unavailable or degraded.
Kernel replica disagreement	Reject protected commits; preserve evidence; reconcile using audit anchors and authorized recovery procedure.
Audit anchor mismatch	Enter Safe Mode for protected mutations; verify lineage and restore or reconcile before admission.

Condition	Kernel behavior
Owner identity uncertainty	Deny protected Owner actions; allow non-protected, separately authorized operation where policy permits.
Constitution version mismatch	Admit only the verified compatible version; restrict affected services and require governed amendment/recovery path.
Host replacement or migration	Re-establish root state, verify recovery material, re-bind host identity, and perform staged runtime admission.

9. Security and Privacy Posture

Kernel credentials and root records use hardware-backed or equivalently protected key material where available. Secrets are referenced rather than exposed to ordinary services. The kernel minimizes retained sensitive payloads: it records evidence identifiers, digests, classifications, and access decisions unless protected content is essential to constitutional recovery or audit. All kernel contracts require authenticated, integrity-protected communication; sensitive payloads remain encrypted to authorized recipients.

10. Kernel Observability

The kernel publishes a bounded Self Model projection: constitutional version, invariant verification state, current mode, protected-policy version, audit-anchor health, pending authorization ceremonies, recovery status, and declared limitations. It does not expose secret material, raw authorization factors, or unnecessary private payloads. Any inconsistency between the projection and protected root state is a constitutional incident.

11. Guarantees

- The kernel **MUST** be the only path for constitutional and other protected commits.
- The kernel **MUST** bind protected decisions to identity, authority, policy, evidence, verification, time, and immutable audit evidence.
- The kernel **MUST NOT** infer protected authorization from conversational language, a prior approval, a cached session, or a network location.
- The kernel **MUST** fail safe for uncertain protected state and **MUST** preserve the evidence necessary for recovery and Owner review.
- The kernel **MUST** expose narrow contracts so governed services remain replaceable without gaining constitutional authority.
- The kernel **MUST** verify continuity before admitting restored, migrated, or partitioned protected state.

12. Constitutional Traceability Audit

Constitutional decisions	Chapter 3 implementation coverage
1–7	Constitutional core boundary, event/audit journal, identity continuity, self-model evidence, governed service contracts.
24–29	Policy enforcement, evidence/trust separation, continuous authentication, introspection, distributed restrictions and recovery.

Constitutional decisions	Chapter 3 implementation coverage
35, 38–43	Protected transactions, supervision boundary, configuration integrity, observability, durability, recovery, safe change.
47–51	Coordination without authority transfer, ownership and succession, sovereignty, verification, self-description and constitutional continuity.
56, 58	Uncertainty-driven restraint, explicit Safe Mode, amendment lifecycle support, and protected invariants.

13. Internal Engineering Review

Check	Result
Constitutional conflicts	None identified. The kernel implements, rather than extends, Book I authority.
Scope discipline	Planning, learning, model execution, and external integration remain outside the kernel.
Authorization safety	Exact-change binding, freshness, liveness, independent factor, and audit requirements are explicit.
Recovery posture	Partition, mismatch, migration, and audit-integrity failure paths enter restriction or Safe Mode.
Owner review required	None. Kernel boundaries and contracts are implementation choices consistent with already locked constitutional requirements.

14. Completion Status

Chapter 3 is complete. It establishes the narrow enforcement boundary that permits later policy, identity, audit, transaction, and recovery services to evolve without diluting constitutional authority. Chapter 4 will specify Identity and Continuity.

Whole-Book Audit Resolution

The following requirements supersede any ambiguous earlier wording in this chapter and are incorporated into this edition:

- The Authority Service may prepare protected decisions, but only the Constitutional Kernel may validate and commit them.
- A recovery constitutional lease is limited to 24 hours. Any extension requires fresh Owner authorization bound to the exact successor, constitutional state, and duration; expiry stops protected canonical mutation and returns the successor to Restricted or Safe Recovery mode.
- An invariant change requires two exact-change Owner Authorization Ceremonies separated by a 72-hour cooling-off period, with a Constitutional Mirror, independently verified recovery point, and explicit continuity classification.